

The screenshot shows a PostgreSQL query plan for a query that took 802 ms to execute. The plan is as follows:

```
1 Result (cost=74571.41..74571.42 rows=1 width=64) (actual time=1260.495..1271.398 rows=1 loops=1)
2   InitPlan 2 (cost=37233.00..37285.71 rows=1 width=32) (actual time=780.900..790.700 rows=1 loops=1)
3     Limit (cost=37233.00..37285.71 rows=1 width=32) (actual time=780.900..790.700 rows=1 loops=1)
4     InitPlan 1 (cost=18698.64..18698.65 rows=1 width=8) (actual time=243.334..252.201 rows=1 loops=1)
5       Aggregate (cost=18698.64..18698.65 rows=1 width=8) (actual time=243.334..252.201 rows=1 loops=1)
6         Finalize GroupAggregate (cost=18534.35..18690.92 rows=618 width=14) (actual time=243.152..252.190 rows=618 loops=1)
7           Group Key: p.product_name
8           Gather Merge (cost=18534.35..18678.56 rows=1236 width=14) (actual time=243.131..252.122 rows=1236 loops=1)
9             Workers Planned: 2
10            Workers Launched: 2
11            Sort (cost=17534.32..17535.87 rows=618 width=14) (actual time=188.494..188.506 rows=95 loops=1)
12              Sort Key: p.product_name
13              Sort Method: quicksort Memory: 28kB
14              Worker 0: Sort Method: quicksort Memory: 28kB
15              Worker 1: Sort Method: quicksort Memory: 28kB
16            Partial HashAggregate (cost=17499.49..17505.67 rows=618 width=14) (actual time=188.494..188.506 rows=618 loops=1)
17              Group Key: p.product_name
18              Batches: 1 Memory Usage: 49kB
19              Worker 0: Batches: 1 Memory Usage: 49kB
20              Worker 1: Batches: 1 Memory Usage: 49kB
21            Hash (cost=16270.33..16270.33 rows=1236 width=14) (actual time=16.970..16.970 rows=1236 loops=1)
```

The screenshot shows a PostgreSQL query plan for the same query after optimization, which took 698 ms to execute. The plan is as follows:

```
1 Aggregate (cost=18782.20..18782.21 rows=1 width=64) (actual time=296.185..302.544 rows=1 loops=1)
2   WindowAgg (cost=18760.59..18772.93 rows=618 width=30) (actual time=296.126..302.515 rows=95 loops=1)
3     Sort (cost=18760.57..18762.12 rows=618 width=22) (actual time=296.121..302.484 rows=95 loops=1)
4       Sort Key: (count(*)), p.product_name
5       Sort Method: quicksort Memory: 28kB
6     WindowAgg (cost=18719.59..18731.93 rows=618 width=22) (actual time=296.039..302.430 rows=95 loops=1)
7       Sort (cost=18719.57..18721.11 rows=618 width=14) (actual time=296.021..302.382 rows=95 loops=1)
8         Sort Key: (count(*)) DESC, p.product_name
9         Sort Method: quicksort Memory: 28kB
10      Finalize GroupAggregate (cost=18534.35..18690.92 rows=618 width=14) (actual time=295.772..302.382 rows=618 loops=1)
11        Group Key: p.product_name
12        Gather Merge (cost=18534.35..18678.56 rows=1236 width=14) (actual time=295.755..302.251 rows=1236 loops=1)
13          Workers Planned: 2
14          Workers Launched: 2
15          Sort (cost=17534.32..17535.87 rows=618 width=14) (actual time=233.346..233.358 rows=95 loops=1)
16            Sort Key: p.product_name
17            Sort Method: quicksort Memory: 28kB
18            Worker 0: Sort Method: quicksort Memory: 28kB
```

Я оптимізував SQL-запит, який працював повільно через багаторівневі вкладені підзапити та повторні обчислення. Для прискорення роботи я використав конструкцію WITH щоб відфільтрувати та об'єднати таблиці лише один раз. Потім я використав віконну функцію ROW\_NUMBER для пошуку мінімальних і максимальних значень без дублювання коду. Також додав індекси на ключові стовпці (order\_date, product\_id, client\_id, status), що дозволило базі даних шукати інформацію швидше, уникаючи повного сканування таблиць. Команда EXPLAIN ANALYZE показала різницю у часі виконання запиту до та після оптимізації.